

Organización de Computadoras

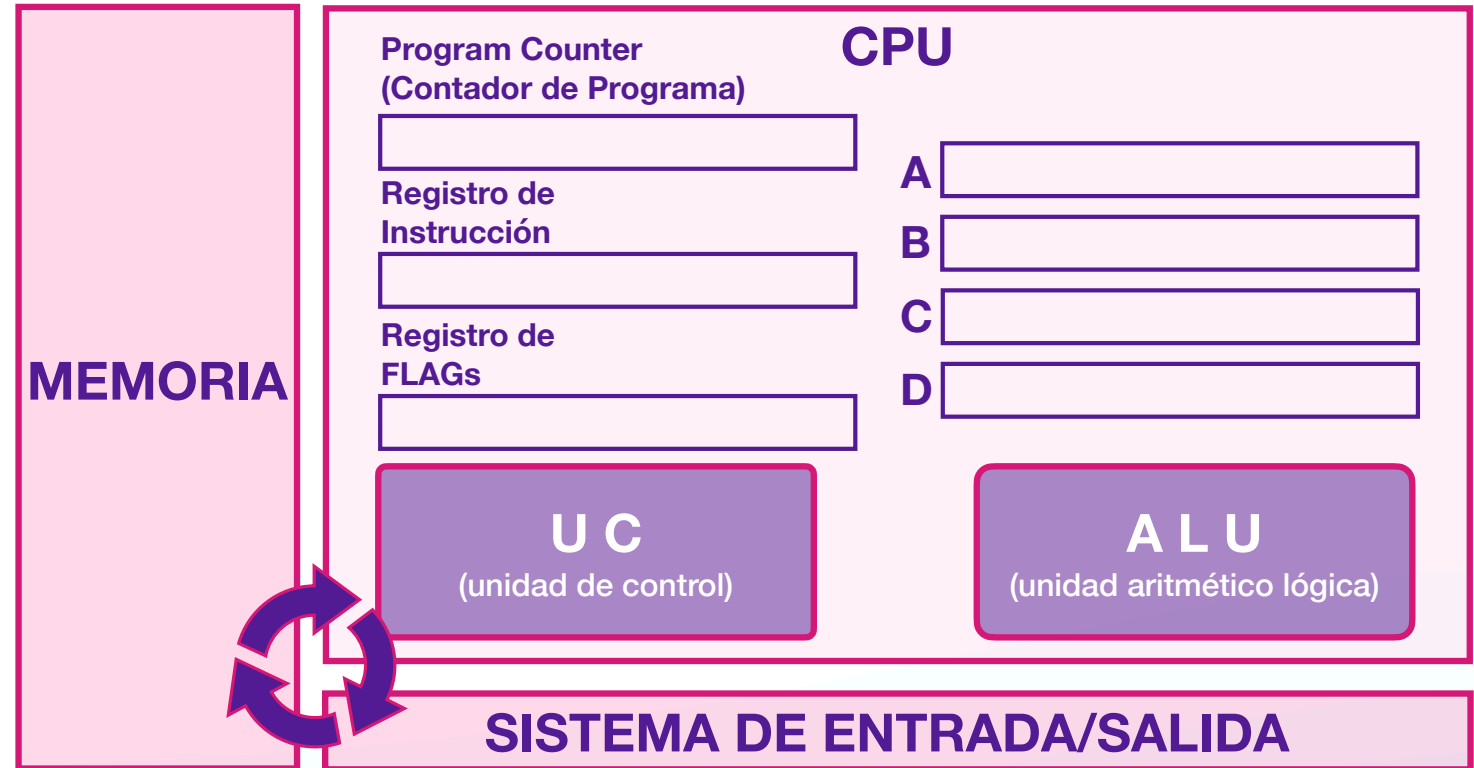
Arquitecturas Microprogramadas

Departamento de Computación - UNRC

El camino a recorrer

- Un poco de Historia y Sistemas Numéricos
- Introducción a la Electrónica
- Representación de Información
- Cómo computar utilizando la electricidad
- Evolución y funcionamiento abstracto de una computadora
- **Arquitecturas microprogramadas**
- Assembly X86
- Eficiencia
 - Pipelines
 - Memoria Caché

Arquitectura von Neumann



John von Neumann: (1903-1957) Físico Matemático Húngaro, Estadounidense. Participó activamente en el proyecto ENIAC y en el desarrollo de la EDVAC, donde propuso el concepto de almacenamiento de programas y datos en la memoria de la computadora, sentando las bases de la arquitectura de von Neumann, que se convirtió en el modelo predominante en las computadoras modernas.

Arquitectura von Neumann

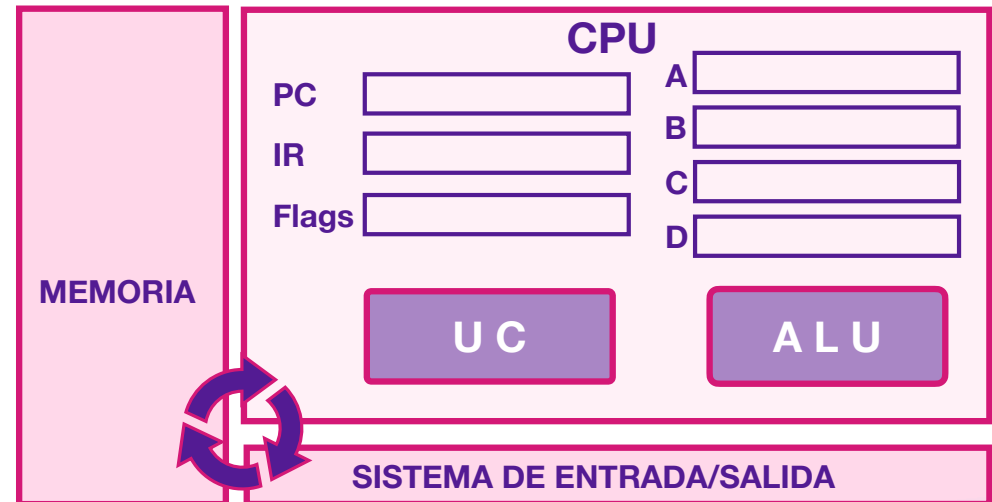
Fetch: La Unidad de control obtiene de la memoria la próxima instrucción que indica el *contador de programa* (**PC**) y la almacena en el Registro de Instrucción (**IR**). Finalmente actualiza el **PC** indicando la dirección de la próxima instrucción.



Decode: La Unidad de control decodifica la instrucción y obtiene de la memoria (si fuere necesario) la información que involucra dicha instrucción.



Execute: La ALU ejecuta (calcula) el resultado de la operación y lo almacena en un registro o memoria.





SOFTWARE

HARDWARE

Conjunto de
Instrucciones (ISA)



x86

ARM

64 bits



RISC-V



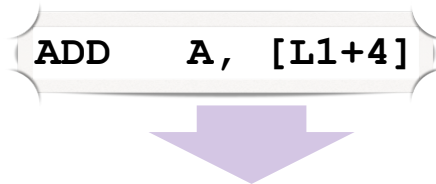
Ejemplo de tareas asociadas a la ejecución de una instrucción

ADD A, [B]



- 1) Recuperar la instrucción de la memoria (al IR)
- 2) Decodificar cuál es la instrucción
- 3) Calcular los operandos (memoria efectiva)
- 4) Recuperar los operandos (en registros)
- 5) Ejecutar la instrucción
- 6) Guardar el resultado

Ejemplo de tareas asociadas a la ejecución de una instrucción



- 1) Recuperar la instrucción de la memoria (al IR)**
 - Configurar la dirección (MAR) con el PC
 - Activar la Lectura de Memoria al IR
 - Incrementar el PC
- 2) Decodificar cuál es la instrucción**
 - Decodificar ADD
- 3) Calcular los operandos (memoria efectiva)**
 - Calcular la dirección L1+4
- 4) Recuperar los operandos (en registros)**
 - Guardar en el RegistroA de la ALU el contenido de EAX
 - Configurar la dirección (MAR) con L1+4 (calculado en 3)
 - Activar la Lectura de Memoria al RegistroB de la ALU Ejecutar la instrucción
- 5) Ejecutar la instrucción**
 - Activar la ALU con la Operación correspondiente
- 6) Guardar el resultado**
 - Transferir al EAX el valor alojado en el registro de salida de la ALU

Control por Hardware vs. Microprogramada

CONTROL POR HARDWARE

- **Diseño más rápido:** En el control por hardware, las señales de control son generadas directamente por circuitos lógicos específicos diseñados para cada instrucción y operación.
- **Menor flexibilidad:** Cambiar o agregar nuevas instrucciones o operaciones requerirá modificar directamente el hardware, lo que puede ser costoso y complejo.
- **Eficiencia en tiempo de ejecución:** el control por hardware puede ser más eficiente en términos de tiempo de ejecución para instrucciones comunes.
- **Mayor complejidad de diseño:** La implementación de un controlador por hardware para cada instrucción puede aumentar la complejidad del diseño del procesador.

CONTROL MICROPROGRAMADO

- **Diseño más flexible:** En el control microprogramado, las instrucciones se implementan utilizando una secuencia de microinstrucciones almacenadas en una memoria especial llamada microprograma. Esto facilita la modificación y la adición de nuevas instrucciones o operaciones sin cambiar el hardware físico.
- **Mayor tiempo de ejecución:** el control microprogramado puede ser más lento en comparación con el control por hardware.
- **Facilidad de actualización:** Las modificaciones y mejoras en las instrucciones se pueden realizar actualizando el microprograma sin necesidad de alterar el hardware subyacente.
- **Menos complejidad de diseño:** El control microprogramado puede simplificar el diseño del hardware al reducir la cantidad de circuitos de control necesarios.

Control por Hardware vs. Microprogramada

SOFTWARE

HARDWARE



Conjunto de Instrucciones (ISA)

Registro de Instrucción (IR)

Unidad de control

(Circuito lógico combinatorial y
secuencial fija)

Señales de
control

Control por hardware

Unidad de control

*memoria interna con
micro-programas*

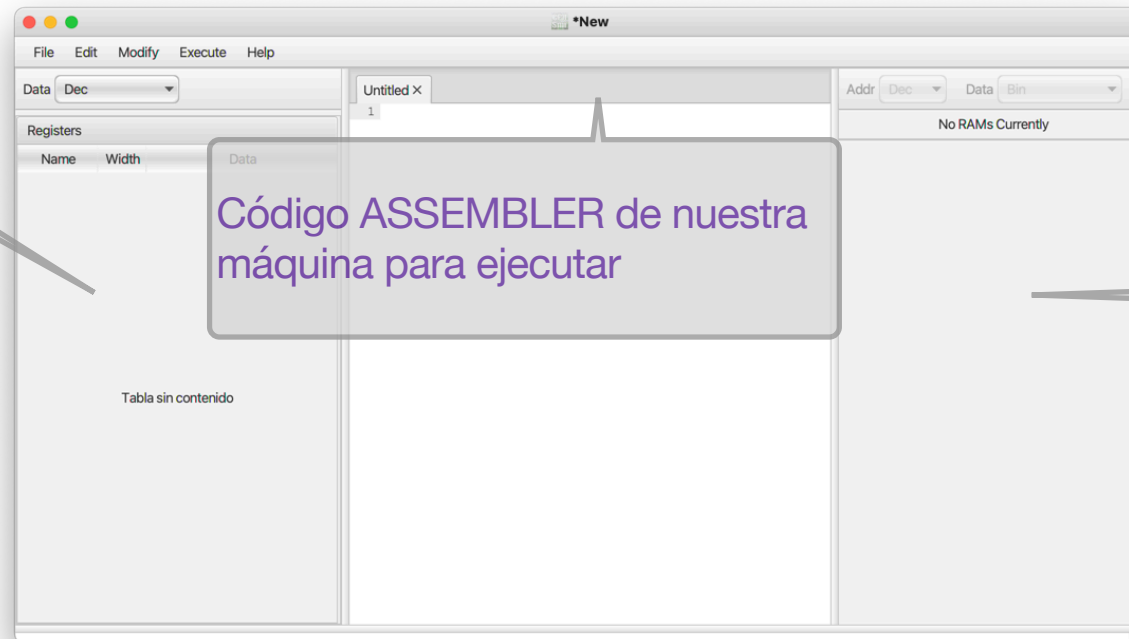
(Circuito lógico combinatorial y
secuencial fija)

Señales de
control

Control micro-programado

Implementando nuestro propio procesador con CPU-SIM

Información de los Registros de nuestro CPU



Información que reside en la/s memoria RAM especificadas

Implementando nuestro propio procesador con CPU-SIM

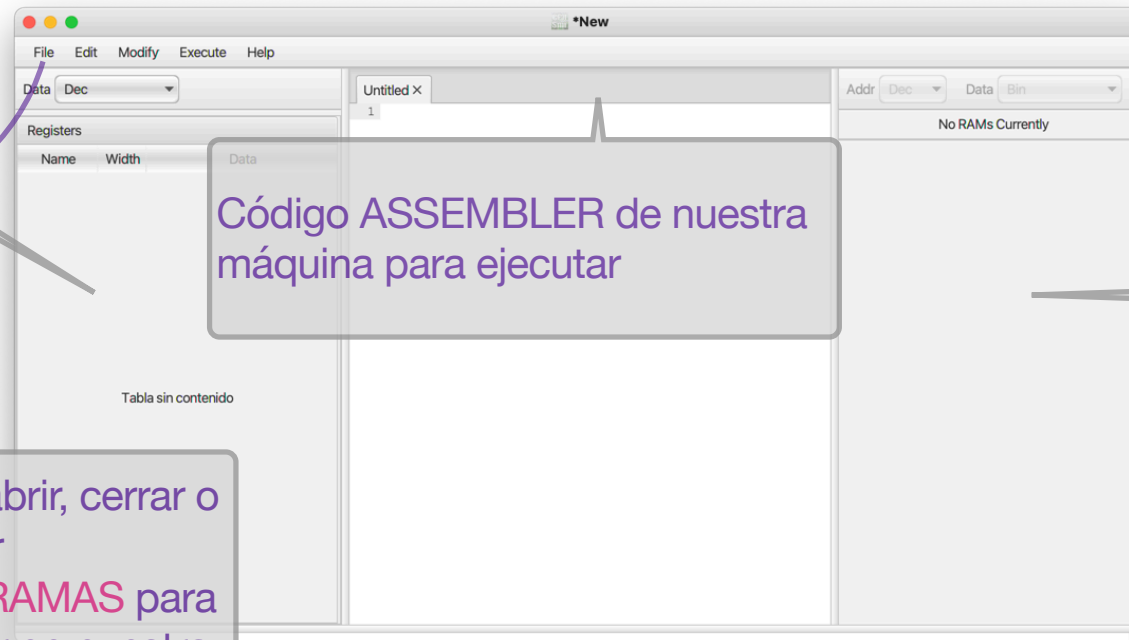
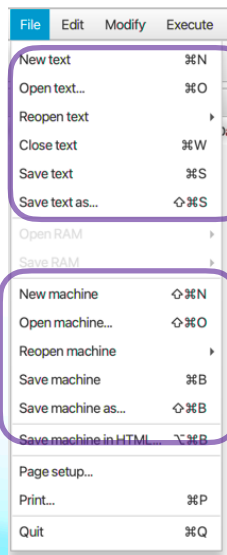
Información de los Registros de nuestro CPU

Código ASSEMBLER de nuestra máquina para ejecutar

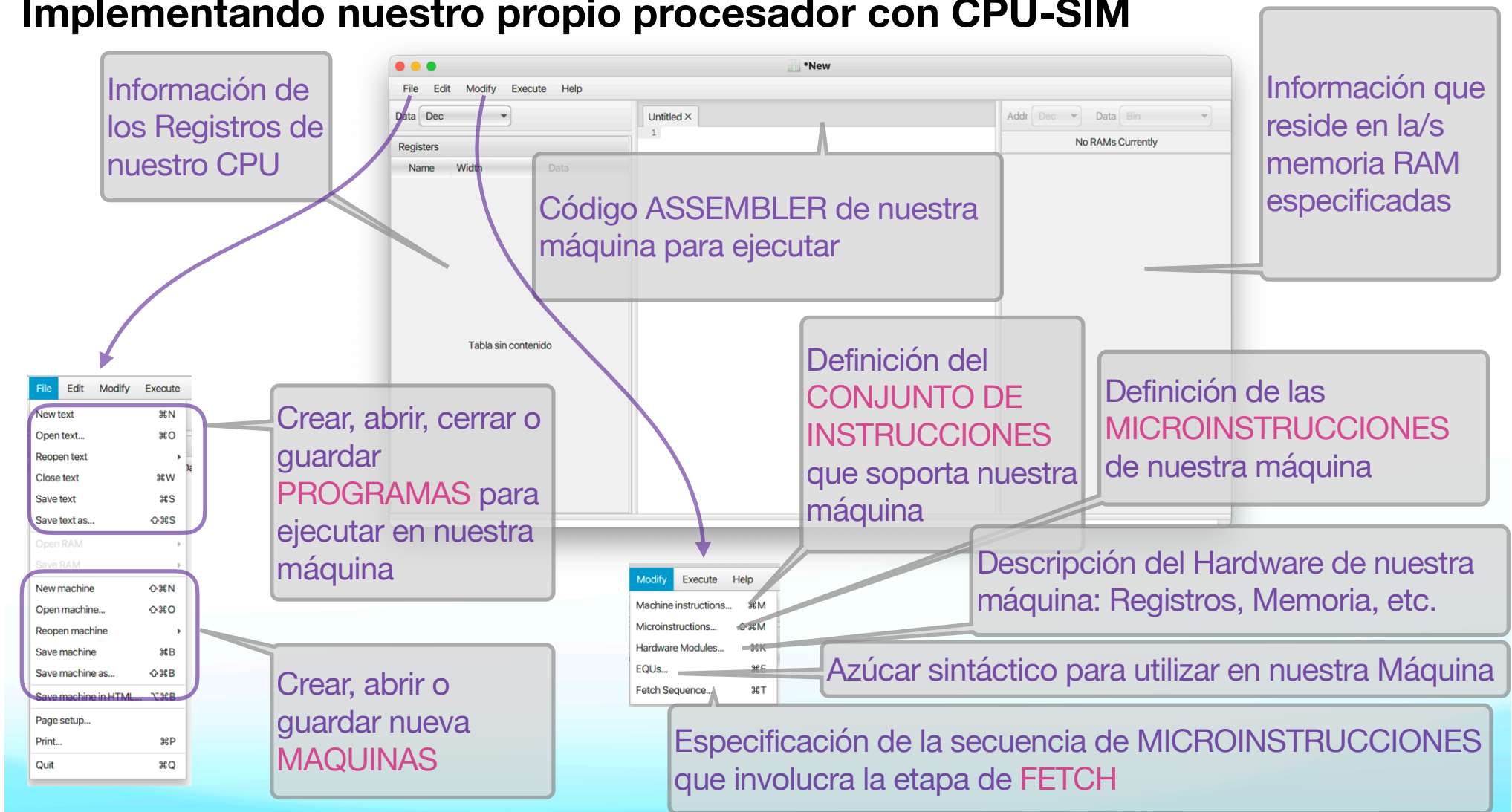
Información que reside en la/s memoria RAM especificadas

Crear, abrir, cerrar o guardar **PROGRAMAS** para ejecutar en nuestra máquina

Crear, abrir o guardar nueva **MAQUINAS**



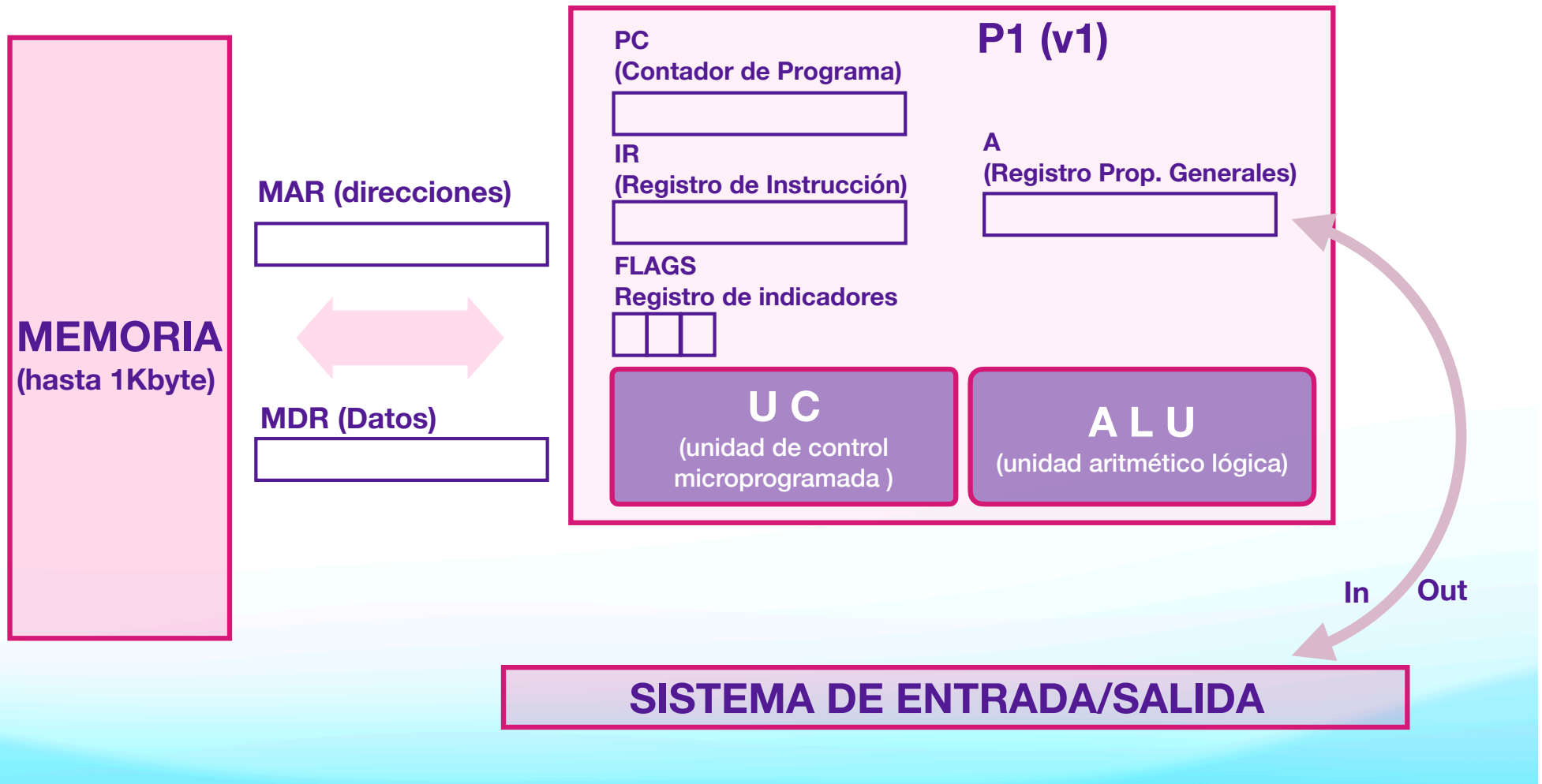
Implementando nuestro propio procesador con CPU-SIM



Implementando nuestro propio procesador con CPU-SIM - Ejemplo P1 (v1)

- Arquitectura de 16 bits
- 1 Registro de Propósito General A
- Program Counter (PC), Registro de Instrucción (IR), Interfaz con la memoria (MAR y MDR) y un registro de 3 Indicadores (FLAGS)
- Hasta 16 Instrucciones:
 - **stop**: Indica la finalización del programa
 - **in**: lee por consola un valor almacenándolo en A
 - **out**: escribe en la consola el valor almacenado en A
 - **load**: lee de la memoria el valor almacenado en la dirección pasada como operando y la guarda en A
 - **save**: similar a load pero almacena el valor en la memoria.
 - **add**: suma el valor almacenado en la dirección pasada como operando al valor actual almacenado en A y guarda el resultado en el A.
 - **jump**: salto incondicional a la instrucción que reside en la dirección que indica el operando
 - **jmpz**: salto a la instrucción que reside en la dirección que indica el operando cuando $A = 0$
 - **jmpn**: salto a la instrucción que reside en la dirección que indica el operando cuando $A < 0$
- Hasta 1Kbyte de memoria RAM

CPUSim - P1 (HARDWARE)



CPUSim - P1 (HARDWARE)

Edit Modules

Type of Module: **Register**

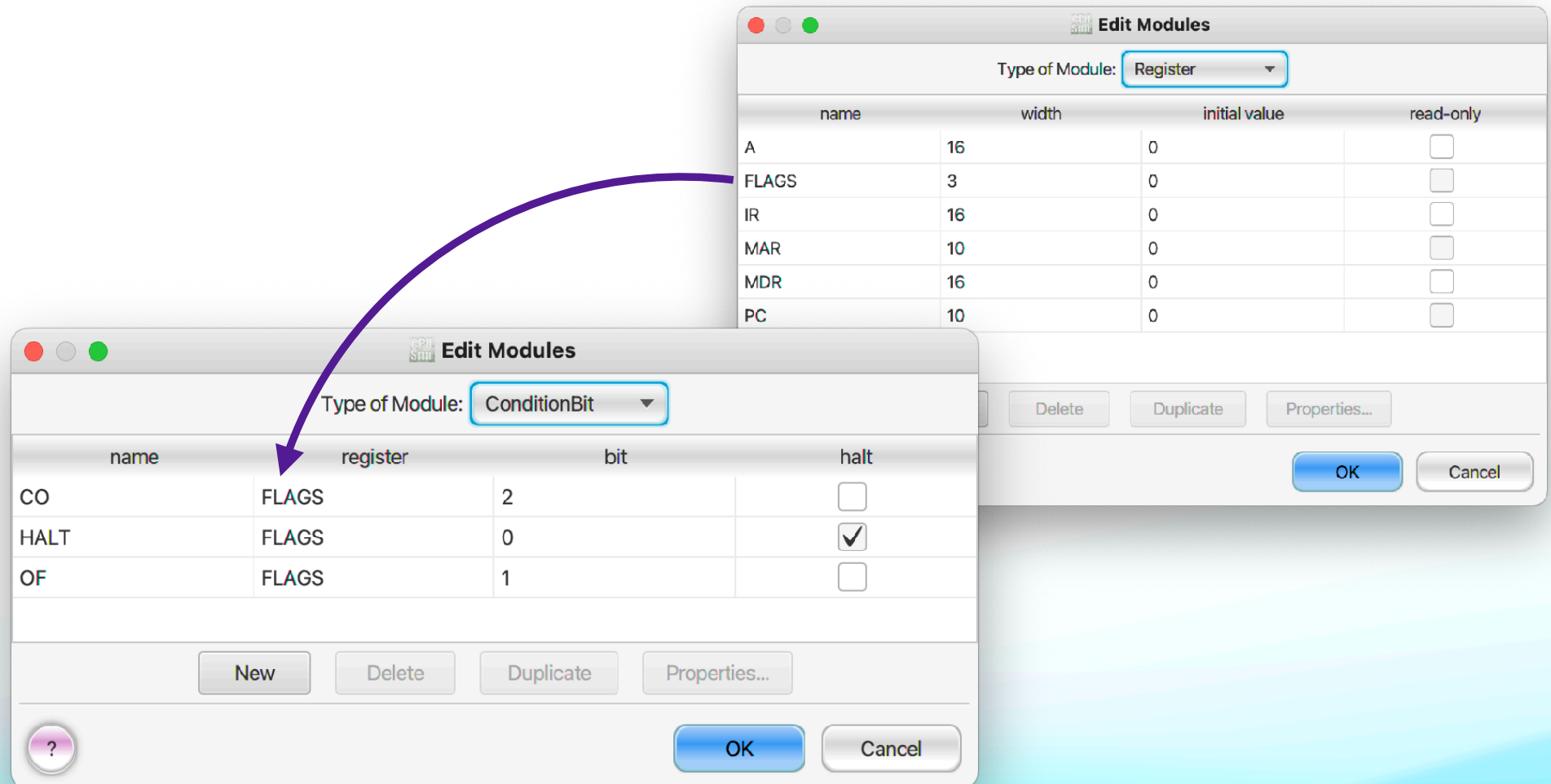
name	width	initial value	read-only
A	16	0	☐
FLAGS	3	0	☐
IR	16	0	☐
MAR	10	0	☐
MDR	16	0	☐
PC	10	0	☐

Edit Modules

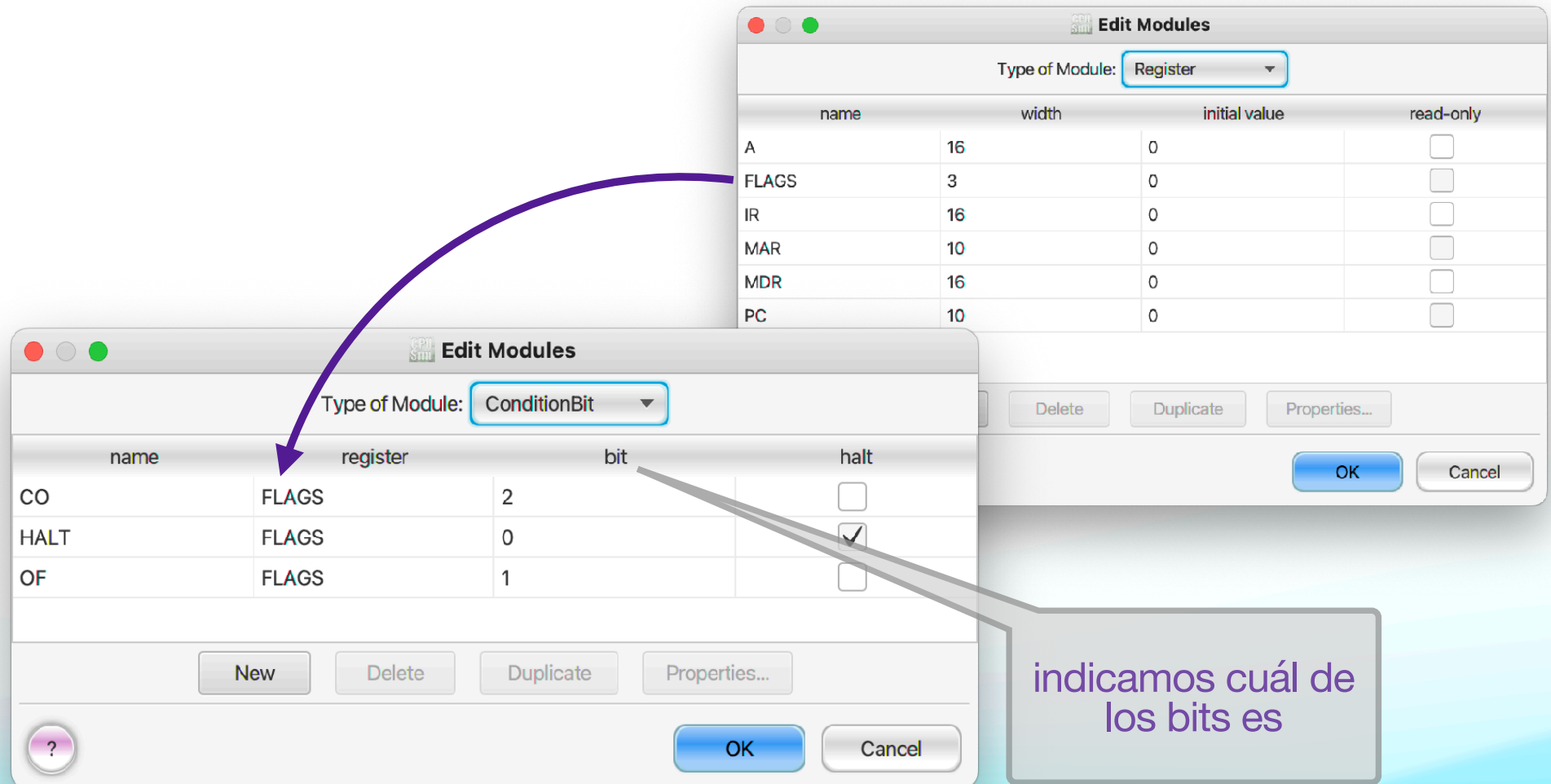
Type of Module: **RAM**

name	length	cellSize
Mem	256	8

CPUSim - P1 (Bits condicionales)



CPUSim - P1 (Bits condicionales)



CPUSim P1 - Instrucciones (ISA) vs Microinstrucciones (hardware)

INSTRUCCIONES

- Es el conjunto de instrucciones que brinda nuestro procesador para ejecutar programas que las utilizan.
- En una arquitectura microprogramada, las Instrucciones están implementadas por medio de microinstrucciones.
- Dependiendo de la arquitectura CISC o RISC es la cantidad de instrucciones que brinda
- La implementación de las mismas (microprogramas) radican en el propio procesador y NO pueden ser modificadas.

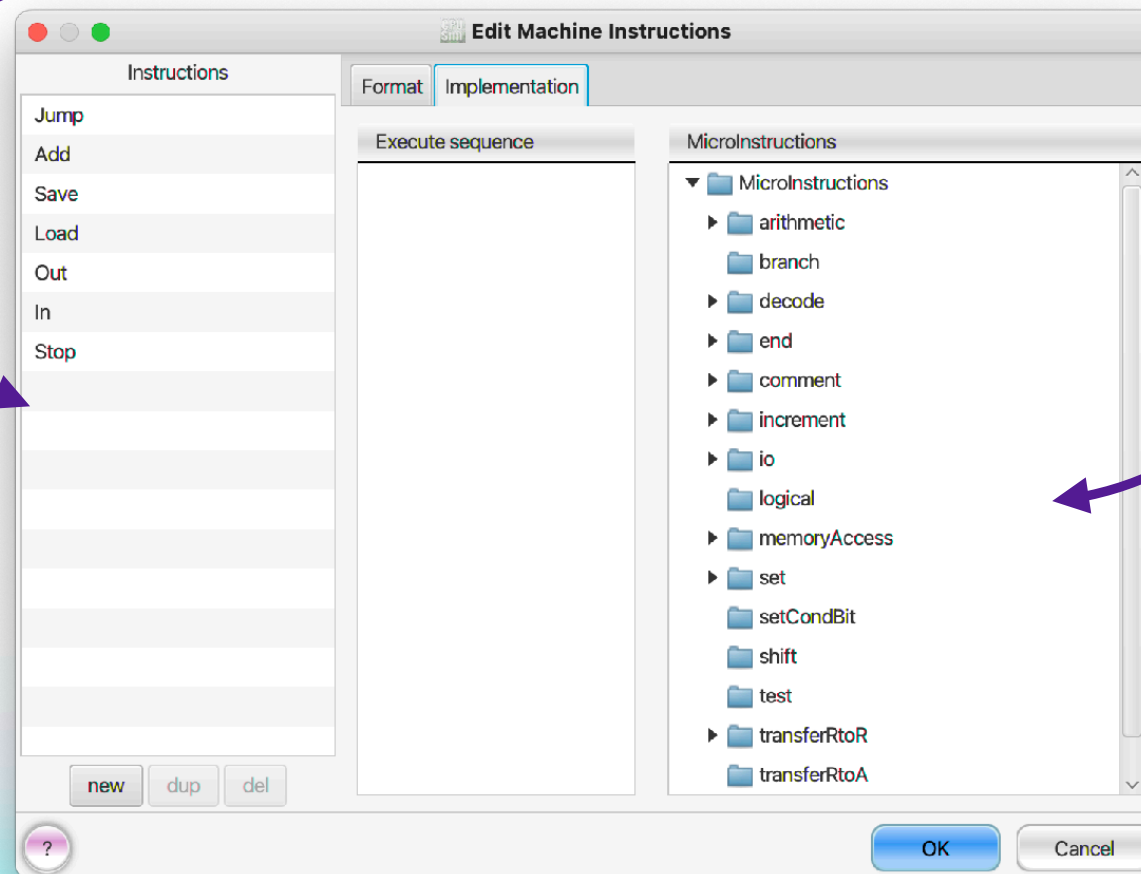
MICROINSTRUCCIONES

- En el control microprogramado, las microinstrucciones están implementadas por medio de hardware
- Cada micro instrucción establece como la acción mínima que puede realizar el procesador, una conexión física en términos de hardware
- En pos de eficiencia y costo, se prioriza la cantidad mínima de microinstrucciones para implementar las Instrucciones

CPUSim P1 - Instrucciones (ISA) vs Microinstrucciones (hardware)

INSTRUCCIONES

MICROINSTRUCCIONES



CPUSim P1 - Implementación del FETCH

- La primera y principal implementación a instrumentar es el FETCH.
- El FETCH es el encargado de traer de la memoria donde reside nuestro programa, la próxima instrucción a ejecutar y almacenarle en el Registro de Instrucción para su decodificación.
- Para implementar el FETCH debemos disponer de las siguientes microinstrucciones:
 - Transferir del “PC” a la interfaz de la memoria “MAR” la dirección
 - Leer la memoria para obtener en la interfaz de datos “MDR” la instrucción
 - Transferir del “MDR” la instrucción al registro de instrucción “IR” para que luego sea decodificado y procesada
 - Incrementar el “PC” para que la próxima instrucción por defecto sea la siguiente a la que estoy ejecutando
 - Indicar la decodificación y correspondiente procesamiento de la instrucción actual

CPUSim P1 - Implementación del FETCH

- Transferir del “PC” a la interfaz de la memoria “MAR” la dirección

name	source	srcStartBit	dest	destStartBit	numBits
PC->MAR	PC	0	MAR	0	10

Los nombres sólo son a efecto de recordar qué hace la microinstrucción a la hora de usarlas

- Leer la memoria para obtener en la interfaz de datos “MDR” la instrucción

name	direction	memory	data	address
MDR->Mem[MAR]	write	Memoria	MDR	MAR

CPUSim P1 - Implementación del FETCH

- Transferir del “PC” a la interfaz de la memoria “MAR” la dirección

CPUSim Edit Microinstructions

Type of Microinstruction: TransferRtoR

name	source	srcStartBit	dest	destStartBit	numBits
PC->MAR	PC	0	MAR	0	10

New Delete Duplicate

OK Cancel

Los nombres sólo son a efecto de recordar qué hace la microinstrucción a la hora de usarlas

- Leer la memoria para obtener en la interfaz de datos “MDR” la instrucción

CPUSim Edit Microinstructions

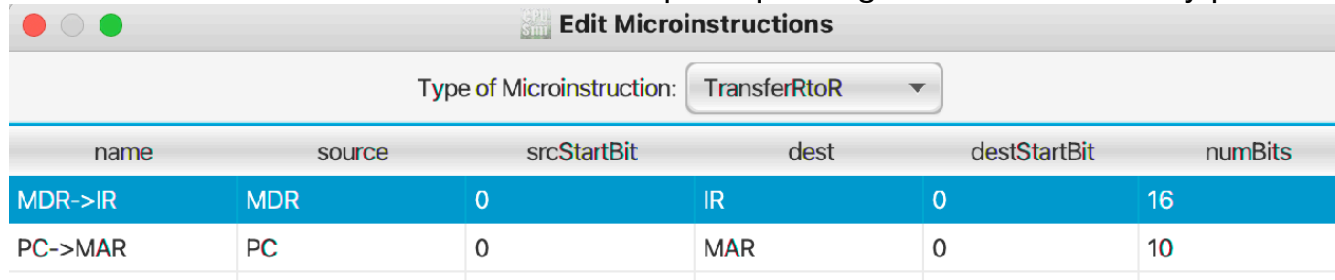
Type of Microinstruction: MemoryAccess

name	direction	memory	data	address
MDR->Mem[MAR]	write	Memoria	MDR	MAR

Los nombres sólo son a efecto de recordar qué hace la microinstrucción a la hora de usarlas

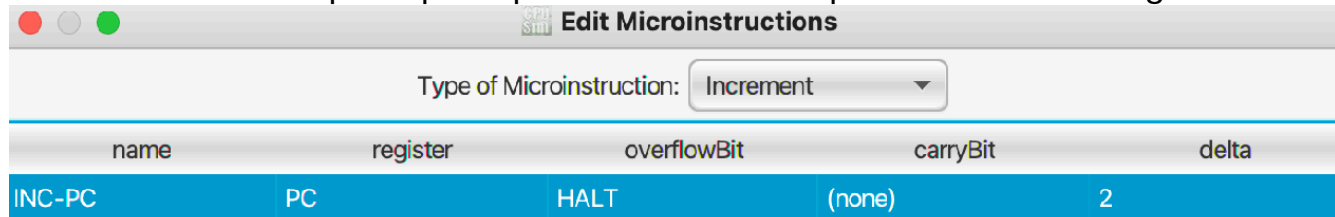
CPUSim P1 - Implementación del FETCH

- Transferir del “MDR” la instrucción al “IR” para que luego sea decodificado y procesada



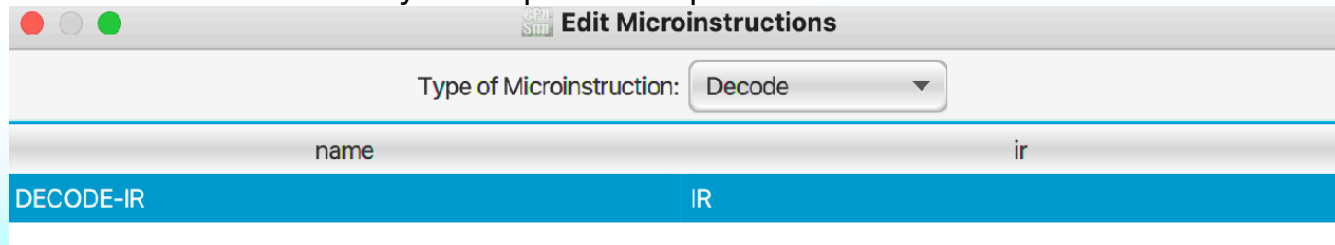
name	source	srcStartBit	dest	destStartBit	numBits
MDR->IR	MDR	0	IR	0	16
PC->MAR	PC	0	MAR	0	10

- Incrementar el “PC” para que la próxima instrucción por defecto sea la siguiente a la que estoy ejecutando



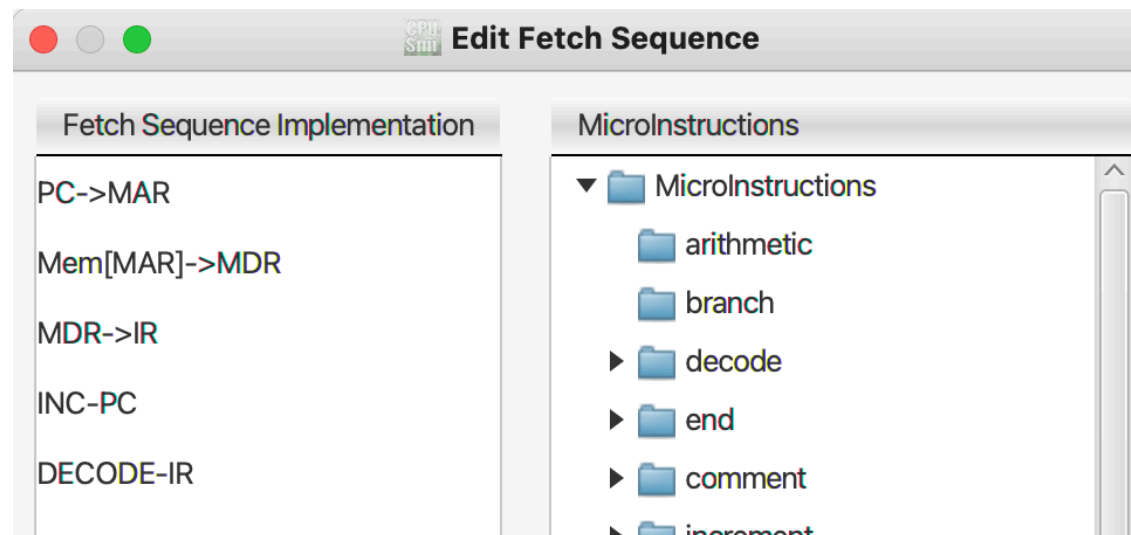
name	register	overflowBit	carryBit	delta
INC-PC	PC	HALT	(none)	2

- Indicar la decodificación y correspondiente procesamiento de la instrucción actual

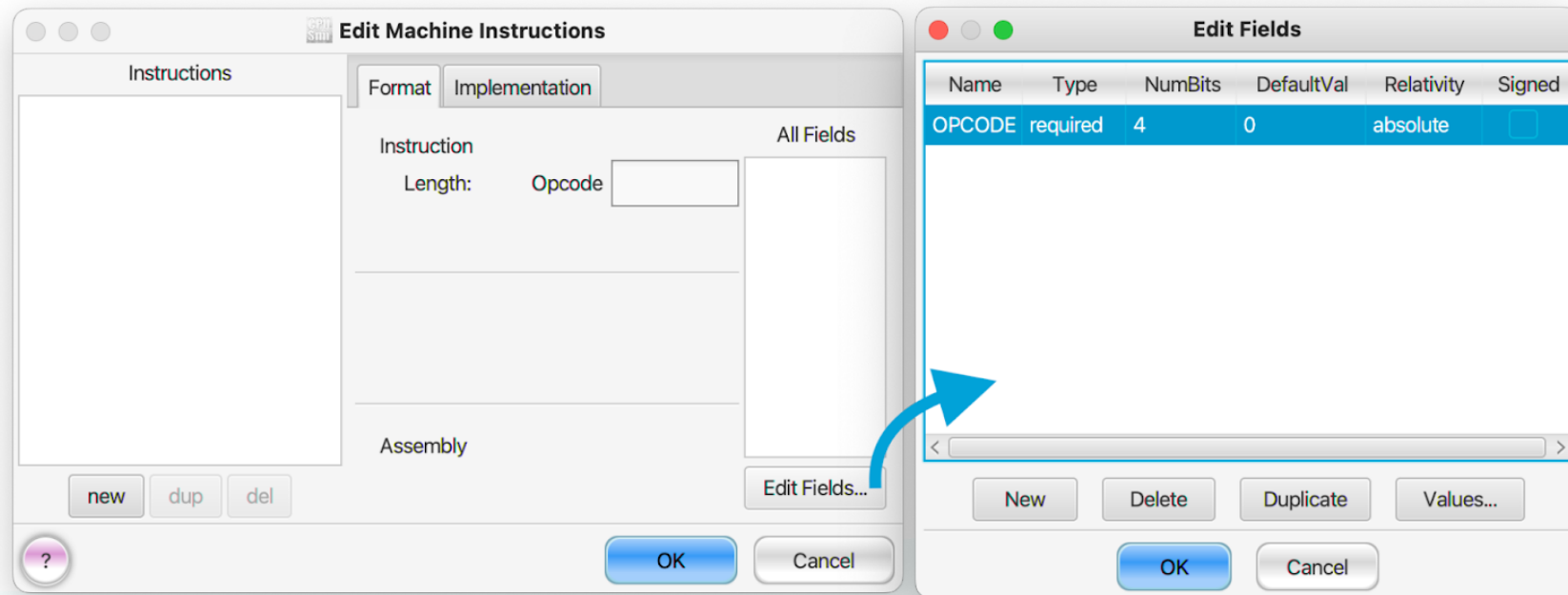


name	ir
DECODE-IR	IR

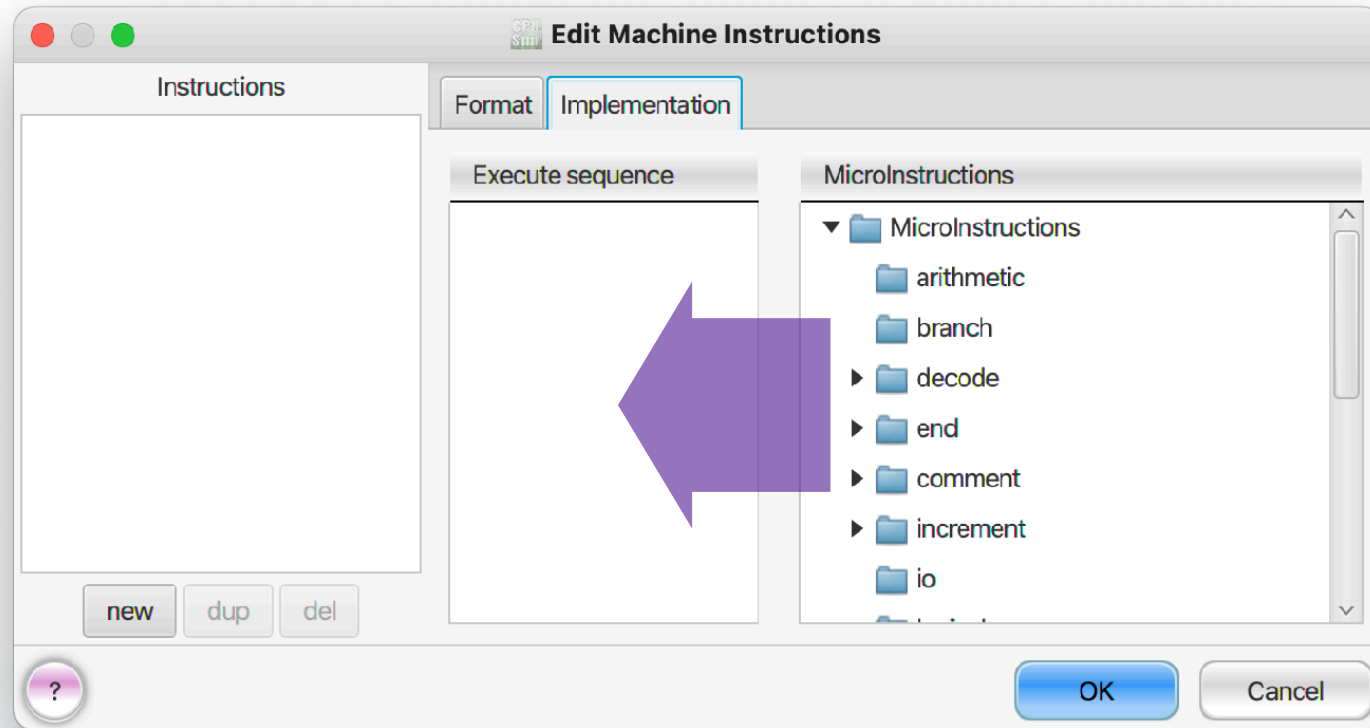
CPUSim P1 - Implementación del FETCH



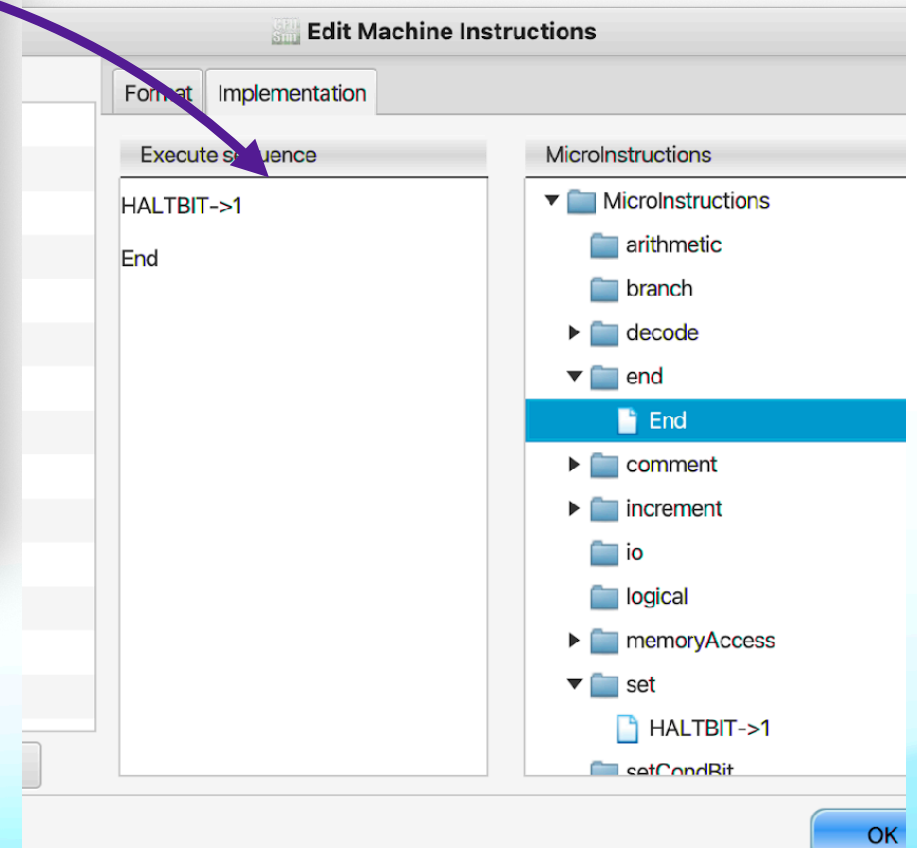
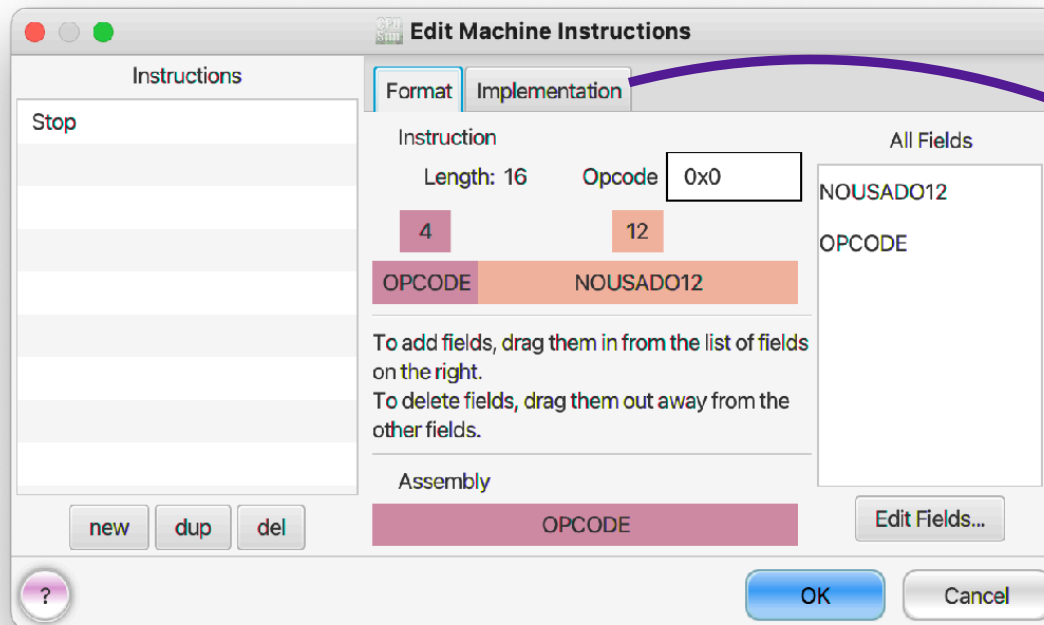
CPUSim P1 - Implementación de instrucciones - Formato y Campos



CPUSim P1 - Implementación de instrucciones - Implementación



CPUSim P1 - Implementación de instrucciones - “Stop”



CPUSim P1 - Probamos nuestro procesador que solo sabe hacer nada :)

The screenshot shows the CPUSim P1 interface. The 'Execute' menu is open, showing options like 'Debug Mode', 'Assemble', 'Assemble & load', 'Assemble, load & run', 'Clear, assemble, load & run', 'Run', 'Stop', 'Reset everything', 'Clear console', and 'Options...'. A blue arrow points from the 'Stop' instruction in the assembly code to the memory dump. The memory dump shows addresses 0 through 9, all containing 0000. A text box at the bottom left says 'Podemos ejecutar paso a paso, en términos de instrucciones o de microinstrucciones'. The bottom right shows the 'Fetch sequence' with steps: PC->MAR, Mem[MAR]->MDR, and MDR->IP.

File Edit Modify **Execute** Help

Data Bin

Registers

Name	Width
A	16
FLAGS	3
IR	16
MAR	10
MDR	16
PC	10

primerPrograma x

```
1 ; Primer programa assembler que no hace nada
2
3 Stop
```

Addr Dec Data Bin

Addr	Data
0	0000 0000
1	0000 0000
2	0000 0000
3	0000 0000
4	0000 0000
5	0000 0000
6	0000 0000
7	0000 0000
8	0000 0000
9	0000 0000

Podemos ejecutar paso a paso, en términos de instrucciones o de microinstrucciones

Go Step by Instr Step by Micro Backup one Instr Backup one Micro Start Over Fetch sequence:

PC->MAR
Mem[MAR]->MDR
MDR->IP

CPUSim P1 - Entrada/salida (un poquito de abstracción)

Microinstrucciones
(hardware I/O) con un
grado de abstracción

Edit Microinstructions			
Type of Microinstruction: IO			
name	type	buffer	direction
A->IO	integer	A	output
IO->A	integer	A	input

Edit Machine Instructions

Instructions

- Jump
- Add
- Save
- Load
- Out
- In**
- Stop

new dup del

Format Implementation

Instruction Length: 16 Opcode 0x1

4 12

OPCODE NOUSADO12

To add fields, drag them in from the list of fields on the right.
To delete fields, drag them out away from the other fields.

Assembly

OPCODE

All Fields

- NOUSADO2
- DIRECCION
- NOUSADO12
- OPCODE

Edit Fields...

OK Cancel

Edit Machine Instructions

Format Implementation

Execute sequence

IO->A

End

MicroInstructions

- MicroInstructions
 - arithmetic
 - branch
 - decode
 - end
 - comment
 - increment
 - io
 - A->IO
 - IO->A
 - logical
 - memoryAccess
 - set

CPUSim P1 - Instrucciones para leer y guardar valores de la memoria

The screenshot shows the CPUSim interface. On the left, a menu lists instructions: Load, Out, In, and Stop. The 'Load' instruction is selected. The main area displays the instruction details: Length: 16, Opcode: 0x3. Below this, the instruction is broken down into fields: OPCODE (4 bits), DIRECCION (10 bits), and NOUSADO2 (2 bits). A purple arrow points from a text box to the DIRECCION field. The text box contains the text: Operando (dirección a la que accedemos). To the right, the 'Type of Microinstruction' is set to 'TransferRtoR'. Below this, a table lists microinstructions:

name	source	srcStartBit	dest	destStartBit	numBit
A->MDR	A	0	MDR	0	16
IR[4..13]->MAR	IR	4	MAR	0	10

The 'Edit Machine Instructions' dialog box is shown. It has two tabs: 'Format' and 'Implementation'. The 'Implementation' tab is selected. On the left, a list of instructions is shown: Add, Save, Load, Out, In, and Stop. The 'Load' instruction is selected. Below this list are buttons for 'new', 'dup', and 'del'. On the right, the 'MicroInstructions' section shows a tree structure with a folder named 'transferRtoR' containing four microinstructions: A->MDR, IR[4..13]->MAR, MDR->A, and MDR->IR. At the bottom of the dialog are 'OK' and 'Cancel' buttons.

CPUSim P1 - Como reservar memoria para valores (Etiquetas)

Etiquetas:

Durante la compilación (Assemble) se reemplazan por el la dirección asignada

Etiquetas:

Podemos utilizarlas como operandos que referencian direcciones de memoria

ej10-secuenciaSumas X

```
1 ;Programa que lee 3 valores y escribe
2 ;el resultado de la suma.
3 In          ;leo de consola un valor en A
4 Add tmp     ;sumo lo que tengo en A+tmp->A
5 Save tmp    ;guardo en tmp el resultado
6 In
7 Add tmp
8 Save tmp
9 In
10 Add tmp
11 Out        ;muestro por pantalla
12 Stop
13
14 tmp: .data 2 0; reserva de 2bytes para valor.
15
```

CPUSim P1 - Tomando el control del secuenciamiento

Fetch Sequence Implementation

PC->MAR

Mem[MAR]->MDR

MDR->IR

INC-PC

DECODE-IR

De manera predeterminada el FETCH incrementa el PC para que la próxima instrucción sea la siguiente.

Cómo podemos brindar una instrucción que indique cuál es la próxima instrucción

```
3 In ;leo de consola un valor en A
4 Add tmp ;sumo lo que tengo en A+tmp->A
5 Save tmp ;guardo en tmp el resultado
6 In
7 Add tmp
8 Save tmp
9 In
10 Add tmp
11 Out ;muestro por pantalla
12 Stop
13
14 tmp: .data 2 0; reserva de 2bytes para valor.
15
```


CPUSim P1 - Tomando el control del secuenciamiento

Fetch Sequence Implementation

PC->MAR

Mem[MAR]->MDR

MDR->IR

INC-PC

DECODE-IR

De manera predeterminada el FETCH incrementa el PC para que la próxima instrucción sea la siguiente.

Cómo podemos brindar una instrucción que indique cuál es la próxima instrucción

```
1  ; Programa que lee 3 valores y escribe el resultado de la suma.
2
3  In           ;leo de consola un valor en A
4  Add tmp     ;sumo lo que tengo en A+tmp->A
5  Save tmp    ;guardo en tmp el resultado
6  In
7  Add tmp
8  Save tmp
9  In
10 Add tmp
11 Out         ;muestro por pantalla
12 Stop
13
14 tmp: .data 2 0; reserva de 2bytes para valor.
15
```

Jump

Format Implementation

Execute sequence

IR[4..13]->PC

End

Format Implementation

Instruction

Length: 16

Opcode 0x6

4

10

2

OPCODE

DIRECCION

NO...

Una instrucción de **salto** "Jump" puede recibir como operando la dirección de la próxima instrucción a ejecutar y modificar el PC con dicha dirección

CPUSim P1 - Tomando el control del secuenciamiento

Utilizamos etiquetas que luego son reemplazadas por la dirección de la instrucción siguiente a la misma.

```
3 leerNum:
4   In      ;leo de consola un valor en A
5   Add tmp ;sumo lo que tengo en A+tmp->A
6   Save tmp ;guardo en tmp el resultado
7   Jump leerNum; salta a leer nuevamente
8   Out     ;muestro por pantalla
9   Stop
10
11 tmp: .data 2 0; reserva de 2bytes para valor
12
```

Quando la instrucción se ejecuta y cambia la dirección del PC con la de la instrucción (siguiente a la etiqueta), alteramos el secuenciamiento predeterminado que establece el FETCH.

CPUSim P1 - Mirando la MATRIX

```

3 leerNum:
4     In          ;leo de consola un valor en A
5     Add tmp     ;sumo lo que tengo en A+tmp->A
6     Save tmp    ;guardo en tmp el resultado
7     Jump leerNum; salta a leer nuevamente
8     Out         ;muestro por pantalla
9     Stop
10
11 tmp: .data 2 0; reserva de 2bytes para valor.
12

```

Instructions

JumpZ

Jump

Add

Save

Load

Out

In

Stop

Format

Implementation

Instruction

Length: 16

Opcode

0x5

4

10

2

OPCODE

DIRECCION

NO...

To add fields, drag them in from the list of fields on the right.

Mem		
	Addr	Data
0	0001	0000
1	0000	0000
2	0101	0000
3	0011	0000
4	0100	0000
5	0011	0000
6	0110	0000
7	0000	0000
8	0010	0000
9	0000	0000
10	0000	0000
11	0000	0000
12	0000	0000
13	0000	0000
14	0000	0000
15	0000	0000
16	0000	0000

CPUSim P1 - Mirando la MATRIX

```
3 leerNum:
4   In      ;leo de consola un valor en A
5   Add tmp ;sumo lo que tengo en A+tmp->A
6   Save tmp ;guardo en tmp el resultado
7   Jump leerNum; salta a leer nuevamente
8   Out     ;muestro por pantalla
9   Stop
10
11 tmp: .data 2 0; reserva de 2bytes para valor.
12
```

Mem		
	Addr	Data
	0	0001 0000
	1	0000 0000
	2	0101 0000
	3	0011 0000
	4	0100 0000
	5	0011 0000
	6	0110 0000
	7	0000 0000
	8	0010 0000
	9	0000 0000
	10	0000 0000
	11	0000 0000
	12	0000 0000
	13	0000 0000
	14	0000 0000
	15	0000 0000
	16	0000 0000

Instructions

JumpZ

Jump

Add

Save

Load

Out

In

Stop

Format

Implementation

Instruction

Length: 16

Opcode 0x5

4

10

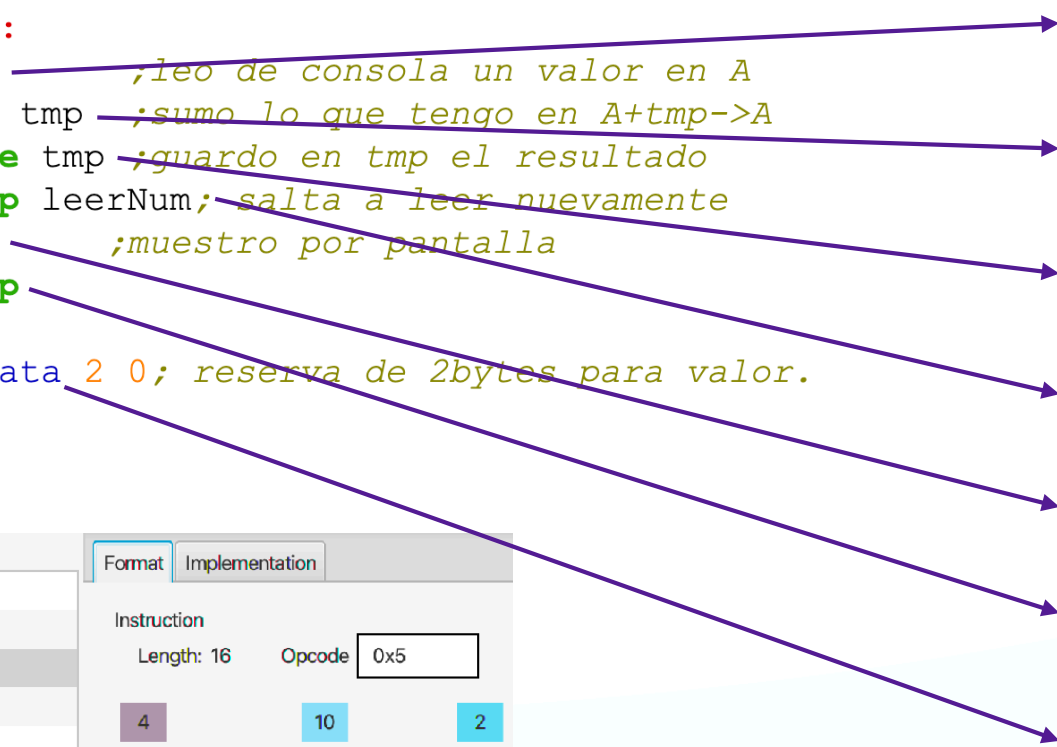
2

OPCODE

DIRECCION

NO...

To add fields, drag them in from the list of fields on the right



CPUSim P1 - Mirando la MATRIX

```

3 leerNum:
4   In      ;leo de consola un valor en A
5   Add tmp ;sumo lo que tengo en A+tmp->A
6   Save tmp ;guardo en tmp el resultado
7   Jump leerNum; salta a leer nuevamente
8   Out     ;muestro por pantalla
9   Stop
10
11 tmp: .data 2 0; reserva de 2bytes para valor.
12

```

Mem		
	Addr	Data
0	0001	0000
1	0000	0000
2	0101	0000
3	0011	0000
4	0100	0000
5	0011	0000
6	0110	0000
7	0000	0000
8	0010	0000
9	0000	0000
10	0000	0000
11	0000	0000
12	0000	0000
13	0000	0000
14	0000	0000
15	0000	0000
16	0000	0000

Instructions	
JumpZ	
Jump	
Add	
Save	
Load	
Out	
In	
Stop	

Format

Implementation

Instruction

Length: 16

Opcode

0x5

4

10

2

OPCODE

DIRECCION

NO...

To add fields, drag them in from the list of fields on the right

Instrucción	OPCode
Stop	0
In	1
Out	2
Save	4
Add	5
Jump	6

CPUSim P1 - Mirando la MATRIX

```

3 leerNum:
4   In      ;leo de consola un valor en A
5   Add tmp ;sumo lo que tengo en A+tmp->A
6   Save tmp ;guardo en tmp el resultado
7   Jump leerNum; salta a leer nuevamente
8   Out     ;muestro por pantalla
9   Stop
10
11 tmp: .data 2 0; reserva de 2bytes para valor.
12

```

Instructions			
JumpZ			
Jump			
Add			
Save			
Load			
Out			
In			
Stop			

Format	Implementation
Instruction	
Length: 16	Opcode: 0x5
4	10
2	
OPCODE	DIRECCION
To add fields, drag them in from the list of fields on the right.	

Instrucción	OPCode
Stop	0
In	1
Out	2
Save	4
Add	5
Jump	6

Mem		
Addr	Data	
0	0001	0000
1	0000	0000
2	0101	0000
3	0011	0000
4	0100	0000
5	0011	0000
6	0110	0000
7	0000	0000
8	0010	0000
9	0000	0000
10	0000	0000
11	0000	0000
12	0000	0000
13	0000	0000
14	0000	0000
15	0000	0000
16	0000	0000

0000001100

12

CPUSim P1 - Mirando la MATRIX

```

3 leerNum:
4   In      ;leo de consola un valor en A
5   Add tmp ;sumo lo que tengo en A+tmp->A
6   Save tmp ;guardo en tmp el resultado
7   Jump leerNum; salta a leer nuevamente
8   Out     ;muestro por pantalla
9   Stop
10
11 tmp: .data 2 0; reserva de 2bytes para valor.
12

```

Instructions			
JumpZ			
Jump			
Add			
Save			
Load			
Out			
In			
Stop			

Format	Implementation
Instruction	
Length: 16	Opcode: 0x5
4	10
2	
OPCODE	DIRECCION
To add fields, drag them in from the list of fields on the right.	

Instrucción	OPCode
Stop	0
In	1
Out	2
Save	4
Add	5
Jump	6

Mem		
Addr	Data	
0	0001	0000
1	0000	0000
2	0101	0000
3	0011	0000
4	0100	0000
5	0011	0000
6	0110	0000
7	0000	0000
8	0010	0000
9	0000	0000
10	0000	0000
11	0000	0000
12	0000	0000
13	0000	0000
14	0000	0000
15	0000	0000
16	0000	0000

0000001100

12

0

CPUSim P1 - Cómo implementar instrucciones que dependan de una CONDICIÓN

Registers					
Name	Width	Data			
A	16	0000	0000	0000	0000
FLAGS	3	000			
IR	16	0000	0000	0000	0000
MAR	10	00	0000	0000	
MDR	16	0000	0000	0000	0000
PC	10	00	0000	0000	

Una CONDICIÓN a nivel de hardware es la definición de un estado particular de alguno de los registros que puede ser verdadero o no.

Podemos definir microinstrucciones para definir estos estados. Por ej, si un FLAG está encendido o si un registro o parte de él tiene algún valor particular.

Edit Microinstructions

Type of Microinstruction: Test

name	register	start	numBits	comparison	value	omission
<A!=0> (1)	A	0	16	NE	0	1

New Delete Duplicate

? OK Cancel

El efecto de ejecutar la microinstrucción es que cuando la comparación se cumple **OMITE** tantas microinstrucciones siguientes como indiquemos.

CPUSim P1 - Implementando un Salto condicional (cuando A es 0)

JumpZ

Format Implementation

Instruction
Length: 16 Opcode: 0x7

4	10	2
OPCODE	DIRECCION	NOU...

Format Implementation

Execute sequence

<A!=0> (1)
IR[4..13]->PC
End

Microinstructions

- test
- <A!=0> (1)
- transferRtoR
- transferRtoA

Cuando A es DISTINTO de 0, omite la siguiente instrucción.

Sólo modifica el PC cuando A es igual a 0.

Edit Microinstructions

Type of Microinstruction: Test

name	register	start	numBits	comparison	value	omission
<A!=0> (1)	A	0	16	NE	0	1

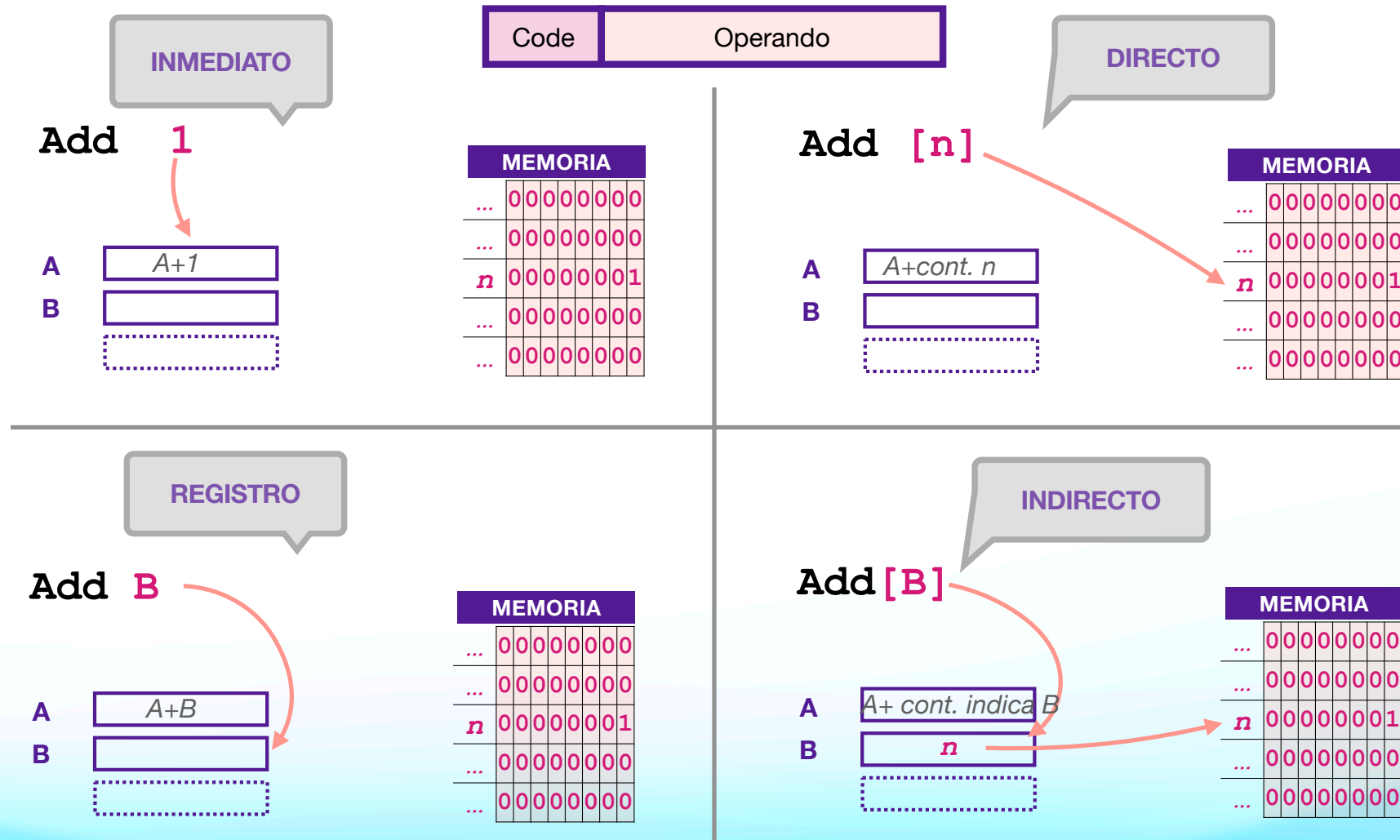
New Delete Duplicate

OK Cancel

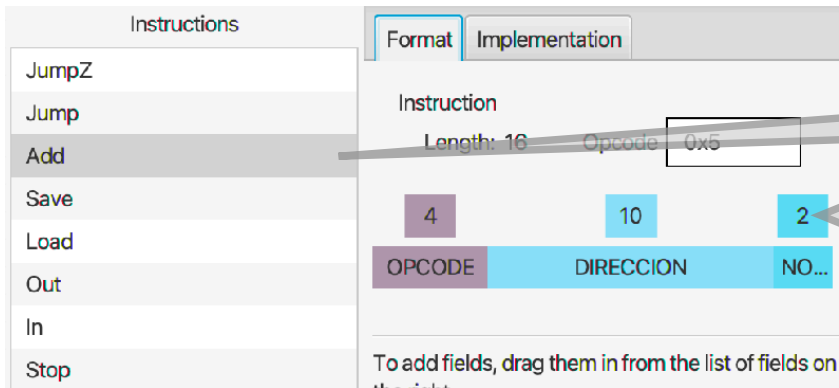
Ejemplo de uso de JumpZ para salir del ciclo cuando ingresan un 0.

```
3 leerNum:
4   In      ;leo de consola un valor en A
5   JumpZ end; si A es 0 salta a end
6   Add tmp ;sumo lo que tengo en A+tmp->A
7   Save tmp ;guardo en tmp el resultado
8   Jump leerNum; salta a leerNum
9 end:
10  Load tmp; cargo la suma acumulada
11  Out     ;muestro por pantalla
12  Stop
13
14 tmp: .data 2 0; reserva de 2bytes para valor.
15
```

CPUSim P1 - Implementando diferentes “tipos” de operandos

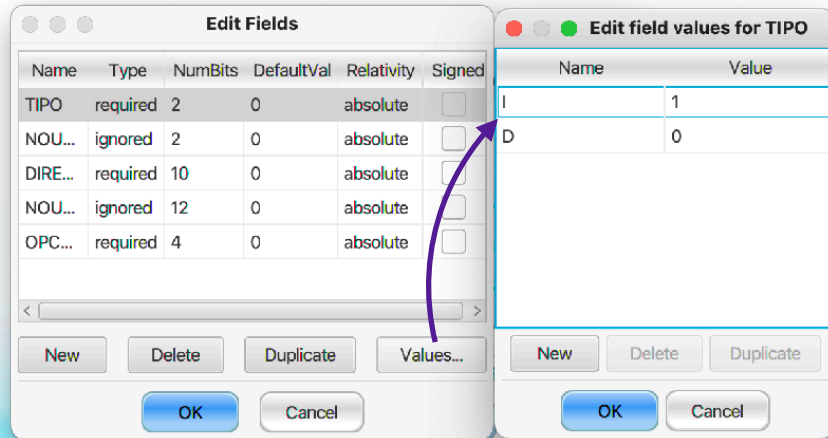


CPUSim P1 - Implementando diferentes “tipos” de operandos (Add)



Cómo podemos diferenciar, por ejemplo entre INMEDIATO y DIRECTO. Lo que tenemos ya implementado es DIRECTO.

Podemos utilizar estos bits que no usamos para diferenciar. Con 2 bits podríamos diferenciar 4 casos. Por ahora sólo vamos a implementar INMEDIATO



Para hacer la programación (usuario de nuestro ASSEMBLY) más “amena”. Podemos definir constantes que podes utilizar para programar pero que luego, cuando se ensamblan y cargan se cargan a memoria se reemplaza por su correspondiente valor. En este caso elegimos “I” para **inmediato** y lo asociamos con el valor 1 y “D” para **directo** y lo asociamos con 0

CPUSim P1 - Implementando diferentes "tipos" de operandos (Add) Directo/Inmediato

Add

Format Implementation

Instruction

Length: 16 Opcode 0x5

4 10 2

OPCODE DIRECCION TIPO

To add fields, drag them in from the list of fields on the right.
To delete fields, drag them out away from the other fields.

Directo (acceso a memoria)

Obtener el valor desde directamente del registro IR

Format Implementation

Execute sequence

//Analizar TIPO

<IR[15]=1>

IR[4..13]->MAR

Mem[MAR]->MDR

MDR+A->A

End

//Inmediato

RESET-MDR

IR[4..13]->MDR

MDR+A->A

End

Type of Microinstruction: Test

name	register	start	numBits	comparison	value	omission
<A!=0> (1)	A	0	16	NE	0	1
<IR[15]=1>	IR	15	1	EQ	1	4

IR

OPCODE 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

Dirección / Valor

Tipo

IR[4..13]->MDR

MDR

0 0 0 0 0 0 6 7 8 9 10 11 12 13 14 15

Valor

name	source	srcStartBit	dest	destStartBit	numBits
A->MDR	A	0	MDR	0	16
IR[4..13]->MAR	IR	4	MAR	0	10
IR[4..13]->MDR	IR	4	MDR	6	10

CPUSim P1 - Implementando diferentes "tipos" de operandos (Add) Directo/Inmediato

Add

Format

Implementation

Instruction

Length: 16 Opcode: 0x5

4

10

2

OPCODE

DIRECCION

TIPO

To add fields, drag them in from the list of fields on the right.
To delete fields, drag them out away from the other fields.

Directo (acceso a memoria)

Obtener el valor desde directamente del registro IR

Qué sucede si queremos sumar inmediatamente un valor negativo:
Add -36 I

Format

Implementation

Execute sequence

//Analizar TIPO

<IR[15]=1>

IR[4..13]->MAR

Mem[MAR]->MDR

MDR+A->A

End

//Inmediato

RESET-MDR

IR[4..13]->MDR

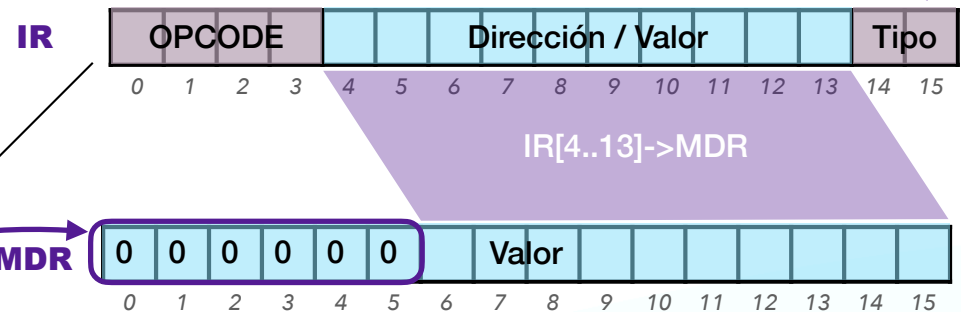
MDR+A->A

End

Omite 4

Type of Microinstruction: Test

name	register	start	numBits	comparison	value	omission
<A!=0> (1)	A	0	16	NE	0	1
<IR[15]=1>	IR	15	1	EQ	1	4



name	source	srcStartBit	dest	destStartBit	numBits
A->MDR	A	0	MDR	0	16
IR[4..13]->MAR	IR	4	MAR	0	10
IR[4..13]->MDR	IR	4	MDR	6	10

CPUsim P1 - Usando diferentes “tipos” de operandos (Add) Directo/Inmediato

```
1 ;Programa que utiliza Jump pero no termina
2
3 leerNum:
4     In          ;leo de consola un valor en A
5     JumpZ end; si A es 0 salta a end
6     Add tmp D ;sumo lo que tengo en A+tmp->A
7     Save tmp ;guardo en tmp el resultado
8     Jump leerNum; salta a leerNum
9 end:
10    Load tmp;cargo la suma acumulada
11    Add 100 I; Sumo 100
12    Out          ;muestro por pantalla
13    Stop
14
15 tmp: .data 2 0; reserva de 2bytes para valor.
```